

Интернационализация в Nuxt.js

Введение

Интернационализация (i18n) — это процесс адаптации приложения для использования на разных языках и в разных регионах. Nuxt.js предоставляет мощные инструменты для создания многоязычных приложений, которые могут автоматически определять язык пользователя и отображать соответствующий контент.

Модуль @nuxtjs/i18n

Основным инструментом для интернационализации в Nuxt.js является модуль `@nuxtjs/i18n`, который предоставляет полный набор функций для создания многоязычных приложений.

Установка

```
npm install @nuxtjs/i18n
```

Базовая настройка

```
// nuxt.config.js
export default {
  modules: [
    '@nuxtjs/i18n'
  ],
  i18n: {
    locales: [
      { code: 'ru', iso: 'ru-RU', file: 'ru.js', name: 'Русский' },
      { code: 'en', iso: 'en-US', file: 'en.js', name: 'English' }
    ],
    defaultLocale: 'ru',
    lazy: true,
    langDir: 'lang/',
    strategy: 'prefix_except_default',
    detectBrowserLanguage: {
      useCookie: true,
      cookieKey: 'i18n_redirected',
      redirectOn: 'root'
    }
  }
}
```

Файлы переводов

```
// lang/ru.js
export default {
  welcome: 'Добро пожаловать',
  hello: 'Привет, {name}!',
  nav: {
    home: 'Главная',
    about: 'О нас',
    contact: 'Контакты'
  }
}

// lang/en.js
export default {
  welcome: 'Welcome',
  hello: 'Hello, {name}!',
  nav: {
    home: 'Home',
    about: 'About',
    contact: 'Contact'
  }
}
```

Использование переводов в компонентах

Базовое использование

```
<template>
  <div>
    <h1>{{ $t('welcome') }}</h1>
    <p>{{ $t('hello', { name: 'Иван' }) }}</p>

    <nav>
      <ul>
        <li><NuxtLink :to="localePath('index')">{{ $t('nav.home') }}</NuxtLink>
      </li>
        <li><NuxtLink :to="localePath('about')">{{ $t('nav.about') }}
      </NuxtLink></li>
        <li><NuxtLink :to="localePath('contact')">{{ $t('nav.contact') }}
      </NuxtLink></li>
      </ul>
    </nav>
  </div>
</template>
```

Переключение языков

```

<template>
  <div>
    <select v-model="$i18n.locale">
      <option v-for="locale in $i18n.locales" :key="locale.code"
:value="locale.code">
        {{ locale.name }}
      </option>
    </select>
  </div>
</template>

```

Локализация маршрутов

Модуль `@nuxtjs/i18n` автоматически создает локализованные маршруты в зависимости от выбранной стратегии.

Стратегии маршрутизации

- **no_prefix:** `/page` (без префикса языка)
- **prefix_except_default:** `/page` (для языка по умолчанию) и `/en/page` (для других языков)
- **prefix:** `/ru/page` и `/en/page` (префикс для всех языков)
- **prefix_and_default:** `/page`, `/ru/page` и `/en/page` (все варианты)

Использование локализованных маршрутов

```

<template>
  <div>
    <NuxtLink :to="localePath('about')">{{ $t('nav.about') }}</NuxtLink>
    <NuxtLink :to="localePath('about', 'en')">About (English)</NuxtLink>
  </div>
</template>

<script>
export default {
  methods: {
    navigateToContact() {
      this.$router.push(this.localePath('contact'))
    }
  }
}
</script>

```

SEO и мета-теги

```

<script>
export default {

```

```

head() {
  return {
    title: this.$t('page.title'),
    meta: [
      {
        hid: 'description',
        name: 'description',
        content: this.$t('page.description')
      }
    ],
    htmlAttrs: {
      lang: this.$i18n.locale
    }
  }
}
}
</script>

```

Плюрализация и форматирование

```

// lang/ru.js
export default {
  items: 'Нет товаров | {count} товар | {count} товара | {count} товаров',
  price: '{price} ₽'
}

// lang/en.js
export default {
  items: 'No items | {count} item | {count} items',
  price: '${price}'
}

```

```

<template>
  <div>
    <p>{{ $tc('items', itemCount, { count: itemCount }) }}</p>
    <p>{{ $t('price', { price: formatPrice(productPrice) }) }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      itemCount: 5,
      productPrice: 1000
    }
  },
  methods: {

```

```

    formatPrice(price) {
      return price.toFixed(2)
    }
  }
}
</script>

```

Интернационализация в Nuxt 3

В Nuxt 3 синтаксис немного изменился:

```

// nuxt.config.ts
export default defineNuxtConfig({
  modules: ['@nuxtjs/i18n'],
  i18n: {
    vueI18n: './i18n.config.ts'
  }
})

```

```

// i18n.config.ts
export default defineI18nConfig(() => ({
  legacy: false,
  locale: 'ru',
  messages: {
    ru: {
      welcome: 'Добро пожаловать'
    },
    en: {
      welcome: 'Welcome'
    }
  }
}))

```

```

<script setup>
const { locale, t } = useI18n()

function switchLanguage(newLocale) {
  locale.value = newLocale
}
</script>

<template>
  <div>
    <h1>{{ t('welcome') }}</h1>
    <button @click="switchLanguage('en')">English</button>
    <button @click="switchLanguage('ru')">Русский</button>
  </div>
</template>

```

```
</div>  
</template>
```

Советы по интернационализации

1. **Используйте ключи вместо строк:** Всегда используйте ключи для переводов, а не жестко закодированные строки
2. **Организируйте переводы по разделам:** Группируйте связанные переводы в объекты для лучшей организации
3. **Учитывайте контекст:** Некоторые слова могут иметь разные переводы в зависимости от контекста
4. **Тестируйте на разных языках:** Убедитесь, что интерфейс корректно отображается на всех поддерживаемых языках
5. **Используйте плюрализацию:** Учитывайте правила множественного числа для разных языков

Заключение

Интернационализация в Nuxt.js с помощью модуля [@nuxtjs/i18n](#) позволяет создавать полностью многоязычные приложения с минимальными усилиями. Правильная настройка переводов, локализованных маршрутов и SEO-оптимизация для разных языков помогут вашему приложению привлечь международную аудиторию.